



Inspiring Excellence

Multi- Level Feedback Queue

We are required to research and explore the CPU allocation fairness issues of Multi-Level Feedback Queue (MLFQ) with special focus on the following areas

- i) Unfair CPU allocation due to I/O – Bound process
- ii) Quantitative Fairness Evaluation

Unfair CPU allocation due to I/O – Bound process

Multi level Feedback queue (MLFQ) scheduling is designed to categorize process dynamically based on their lengths of CPU bursts time. The principal idea of MLFQ is to separate process according to the observed execution characteristics and involves having a set of queues arranged by priority which allows processes to migrate between the queues. The migration is the mechanism MLFQ uses to learn a process behavior and change over time. An entering process is placed in the highest priority queue which is assigned with a small time quantum. If a process consumes its entire time quantum without finishing the burst time the scheduler assumes it is a CPU -intensive task and demotes in to a lower priority queue. Any newly arrived or I/O waiting processes receive higher priority than the ones in lower queues.

The I/O Bound Process tend to monopolize higher priority queues while CPU-bound processes are often demoted to lower levels. The I/O Bound processes spend most of their time waiting and thus have short CPU bursts. When it is given in the CPU it is often less than the time quantum thus they don't get demoted. After I/O request completion , they are returned to high priority

ready state ,ready to preempt any lower -priority process currently running .On the other hand CPU -bound focuses more on computation rather than I/O and typically run until their assigned time quantum expires. After that the scheduler preempts them and demotes them to a lower priority queue.

The imbalance can lead to starvation or unfair CPU allocation among processes. While prioritizing I/O bound processes improves system response for interactive users. This occurs because high-priority queues must be completely empty before the scheduler moves to lower-priority ones. When there is a constant arrival of new high priority (I/O -bound) process,they monopolize the CPU and as a result the scheduler doesn't reach the lower priority queues .

The possible fairness- improving mechanisms with suitable examples have been discussed below;-

- i) **Aging (Promotion)** : This method prevents starvation by gradually increasing the priority of the processes that wait for a long time. In MLFQ for example if a cpu - bound process goes to Queue 2 (FCFS queue) on aging mechanism would eventually move it back to Queue 1 .
- ii) **Priority Boosting** : This mechanism temporarily elevates a process's priority when a specific event happens generally when it finishes an I/O operation .For example in a Windows system a threads priority is boosted when it wakes from I/O ,thus I/O bound and interactive tasks run immediately which keeps devices busy and User Interface responsive.This boost gives MLFQ by extra preference to latency driven I/O
- iii) **Time Quantum Arrangement** : This approach assigns larger time slices to lower priority queues. For example if Queues 0 uses 10 seconds ,Queue 1 might use 20

seconds .The design ensures that once a CPU- bound process reaches a lower queue it is allowed to execute for a relatively long time frame .

Quantitative Fairness Evaluation:

Fairness in CPU scheduling refers to the principle that the operating system, acting as a resource allocator, must distribute resources like CPU time among the processes efficiently and fairly. While maximizing CPU utilization and throughput are key goals, fairness often refers to decreasing the maximum waiting time or response time variance to guarantee good service for all users. Multi-queue (MLFQ) scheduling systems risk starvation where low-priority processes might never run because they are designed to always favor higher-priority tasks. To ensure **fairness** and prevent this indefinite blocking, mechanisms like aging are added, which guarantee that even low-priority processes eventually get a chance to execute.

We introduce a Fairness Index which can serve as a metric to quantitatively assess how evenly CPU time (T_i) is distributed among processes:

$$\text{Fairness Index} = \frac{(\sum T_i)^2}{n \times \sum T_i^2}$$

Here T_i represents the CPU time allocated to process i . The index yields a value between 0 and 1, where 1 indicates perfect fairness (meaning all processes received equal CPU time).

The demonstration of how the fairness index will evaluate on the sample MFLQ workload has been shown below :-

We consider a scenario involving two long-running processes, one I/O-bound and one CPU-bound, running under a non-aging MLFQ policy. Let the total time observed be 50 s

P_A : I/O-bound process. Due to rapid I/O yielding, it quickly returns to the high-priority queue and is favored by the MLFQ scheduler

P_B : CPU-bound process. Due to long CPU bursts, it is immediately demoted to the lowest-priority queue and is largely starved

Process	Type	CPU Time Allocated (T_i)
P_A	I/O Bound	40 s
P_B	CPU Bound	10 s
Total		$\sum T_i = 50$ s

In this scenario: $n = 2$ processes

1. **Calculate** $\sum T_i$ and $(\sum T_i)^2$:

$$\sum T_i = 40 + 10 = 50 \text{ s}$$

$$(\sum T_i)^2 = 2500 \text{ s}$$

2. Calculate $\sum T_i^2$:

$$40^2 + 10^2 = 1600 + 100 = 1700 \text{ s}$$

3. **Calculate Fairness Index:**

$$\text{Fairness Index} = 2500 / (2 \times 1700) = 2500 / 3400 \approx 0.735$$

The resulting index value of approximately 0.735 reflects the clear bias toward the I/O-bound process (P_A) On the other hand if we split the time evenly ($T_A = 25$ & $T_B = 25$), the index would be

$$50^2 / (25^2 + 25^2) = 1$$

Which represents perfect fairness.

We are required to analyze how changing the number of queues, quantum lengths or aging intervals affects fairness scores and CPU distribution .

Feature	Allocation	Fairness Index
Queue No.	A higher number of queues allows for finer priority differentiation (Multilevel Queue Scheduling). CPU-bound jobs would have more steps to descend, reducing initial demotion.	Without a fundamental change in the way the lowest priority tasks end up being ultimately executed (such as the introduction of aging in a new queue), the index will still be dominated by the inherent priority bias toward short bursts times
Quantum Length	If the quantum length is increased significantly, MLFQ starts to resemble FCFS scheduling	Fairness index will increase
Aging Interval	Aging is the primary mechanism used to prevent starvation by promoting low-priority processes that wait too long	Fairness index will also increase

The goal in tuning MLFQ is often not to achieve perfect quantitative fairness (FI = 1.0) but to balance priority in order to get a better responsiveness. That's why aging is necessary addition in this algorithm